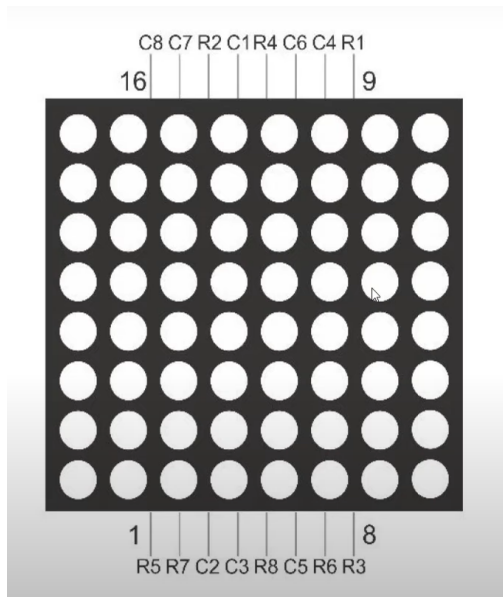


JUEGO DE LA SERPIENTE



Para crear el juego de la serpiente con la matriz LED de 8x8, el módulo joystick en una placa Arduino UNO, primero, presentaremos el esquema de conexión y luego el código en Arduino.

Esquema de Conexión:

Matriz LED 8x8:

Componente	Pin Matriz LED	Pin Arduino
Columna 1	C1	3
Columna 2	C2	10
Columna 3	C3	11
Columna 4	C4	6
Columna 5	C5	13
Columna 6	C6	5
Columna 7	C7	1
Columna 8	C8	0
Fila 1	R1	7
Fila 2	R2	2
Fila 3	R3	A0
Fila 4	R4	4
Fila 5	R5	8
Fila 6	R6	A1

Componente	Pin Matriz LED	Pin Arduino
Fila 7	R7	9
Fila 8	R8	12

Joystick:

- GND a GND del Arduino.
- 5V a 5V del Arduino.

Componente	Pin Joystick	Pin Arduino
Eje X	VRX	A2
Eje Y	VRY	A3
Botón	SW	A4

Código:

```
#define FILAS 8
#define COLUMNAS 8

// Definición de pines para las columnas
const int pinesColumnas[COLUMNAS] = {3, 10, 11, 6, 13, 5, 1, 0};

// Definición de pines para las filas
const int pinesFilas[FILAS] = {7, 2, A0, 4, 8, A1, 9, 12};

// Pines del joystick
#define JOYSTICK_X A2
#define JOYSTICK_Y A3
#define JOYSTICK_SW A4

int direccion = 0; // 0: Derecha, 1: Abajo, 2: Izquierda, 3: Arriba
int velocidad = 500; // Velocidad inicial (ms)

// Posición inicial de la serpiente
int serpienteX[64] = {4, 3, 2};
int serpienteY[64] = {4, 4, 4};
int longitudSerpiente = 3;

// Posición de la manzana
int manzanaX = 0;
int manzanaY = 0;

void setup() {
  Serial.begin(9600);
```

```

pinMode(JOYSTICK_SW, INPUT_PULLUP);

// Configurar pines de columnas y filas como salidas
for (int i = 0; i < COLUMNAS; i++) {
    pinMode(pinesColumnas[i], OUTPUT);
}
for (int i = 0; i < FILAS; i++) {
    pinMode(pinesFilas[i], OUTPUT);
}

randomSeed(analogRead(0));
generarManzana();
}

void loop() {
    leerJoystick();
    moverSerpiente();
    mostrarMatriz();
    delay(velocidad);
}

void leerJoystick() {
    int x = analogRead(JOYSTICK_X);
    int y = analogRead(JOYSTICK_Y);

    if (x < 300) {
        direccion = 2; // Izquierda
    } else if (x > 700) {
        direccion = 0; // Derecha
    } else if (y < 300) {
        direccion = 3; // Arriba
    } else if (y > 700) {
        direccion = 1; // Abajo
    }
}

void moverSerpiente() {
    int nuevaX = serpienteX[0];
    int nuevaY = serpienteY[0];

    switch (direccion) {
        case 0: nuevaX++; break; // Derecha
        case 1: nuevaY++; break; // Abajo
        case 2: nuevaX--; break; // Izquierda
        case 3: nuevaY--; break; // Arriba
    }

    // Verificar colisión con bordes
    if (nuevaX < 0 || nuevaX >= COLUMNAS || nuevaY < 0 || nuevaY >= FILAS) {
        gameOver();
    }

    // Verificar colisión consigo misma

```

```

for (int i = 0; i < longitudSerpiente; i++) {
    if (serpienteX[i] == nuevaX && serpienteY[i] == nuevaY) {
        gameOver();
    }
}

// Mover cuerpo de la serpiente
for (int i = longitudSerpiente; i > 0; i--) {
    serpienteX[i] = serpienteX[i - 1];
    serpienteY[i] = serpienteY[i - 1];
}

// Mover cabeza de la serpiente
serpienteX[0] = nuevaX;
serpienteY[0] = nuevaY;

// Verificar si ha comido la manzana
if (nuevaX == manzanaX && nuevaY == manzanaY) {
    longitudSerpiente++;
    velocidad = max(100, velocidad - 50); // Aumentar la velocidad (disminuir
el delay)
    generarManzana();
}
}

void generarManzana() {
    bool manzanaValida = false;
    while (!manzanaValida) {
        manzanaX = random(0, COLUMNAS);
        manzanaY = random(0, FILAS);
        manzanaValida = true;
        for (int i = 0; i < longitudSerpiente; i++) {
            if (serpienteX[i] == manzanaX && serpienteY[i] == manzanaY) {
                manzanaValida = false;
                break;
            }
        }
    }
}

void mostrarMatriz() {
    // Apagar todas las filas
    for (int i = 0; i < FILAS; i++) {
        digitalWrite(pinesFilas[i], LOW);
    }

    // Encender la matriz
    for (int i = 0; i < FILAS; i++) {
        for (int j = 0; j < COLUMNAS; j++) {
            if (haySerpiente(j, i) || hayManzana(j, i)) {
                digitalWrite(pinesColumnas[j], LOW);
            } else {
                digitalWrite(pinesColumnas[j], HIGH);
            }
        }
    }
}

```

```

    }
}
digitalWrite(pinesFilas[i], HIGH);
delayMicroseconds(100); // Pequeña demora para actualizar la pantalla
digitalWrite(pinesFilas[i], LOW);
}
}

bool haySerpiente(int x, int y) {
    for (int i = 0; i < longitudSerpiente; i++) {
        if (serpienteX[i] == x && serpienteY[i] == y) {
            return true;
        }
    }
    return false;
}

bool hayManzana(int x, int y) {
    return (manzanaX == x && manzanaY == y);
}

void gameOver() {
    // Mostrar mensaje de Game Over
    while (1); // Bucle infinito para detener el juego
}

```

Descripción del Código:

- **Configuración de Pines:** Asignación de pines digitales y analógicos del Arduino para las filas y columnas de la matriz LED según tus especificaciones.
- **Leer el Joystick:** Lectura de los valores analógicos de los ejes X e Y del joystick para determinar la dirección de movimiento.
- **Mover la Serpiente:** Actualización de las posiciones de la serpiente y verificación de colisiones.
- **Generar Manzana:** Generación de coordenadas aleatorias para la manzana, asegurando que no aparezca sobre la serpiente.
- **Mostrar Matriz:** Actualización de la matriz LED encendiendo los LEDs correspondientes a la serpiente y la manzana.
- **Aumentar la Velocidad:** Incremento de la velocidad del juego cada vez que la serpiente come una manzana disminuyendo el delay.
- **Game Over:** Detención del juego si la serpiente colisiona con los bordes o consigo misma.